FieldMind: team handoff

State as of Saturday 3 October 2026, about 23:15 IST

Read this first

> **The short version**
>
> Work is going on in two places at once, on two git branches.
>
> **Fedora laptop + QIDK board, branch `main`:** we built four small language models in one identical way, put them on the board, and started a long unattended run (the "campaign") that picks the model and measures it. The first attempt failed three times this evening. The cause was found (the model server filled the board's memory) and fixed, along with a bug in our runner. **A fresh campaign, starting again from screening, was being launched at about 23:10 to run overnight.** Section 5.5 has the story and how to check on it.
>
> **Mac, branch `multi-agent`:** the multi-agent system. Phase 1 (plumbing) is finished and tagged. Phase 2 (making the prompts small) is half done: 4 of 8 commits are in.
>
> **You do not need the board to continue the multi-agent work.** It is built and tested against a fake model (the "mock") on any laptop. Only the Fedora laptop can run real models.

Where a statement in this document is something we were told but did not check, or something still open, it says so. Section 10 lists those in one place.

1. The project in simple words

FieldMind (Embedded Systems Workshop, Lab 2, Project 4; advisor Prof. Priyesh Shukla, TA Aryama Murthy; team of 4) watches one industrial boiler using a phone-class chip with no internet. Every 30 seconds it reads 6 sensor values, decides whether something is wrong, names the likely cause and suggests what an engineer should check. It only advises. It never controls the plant.

- **The boiler:** a 67 TPH coal-fired fluidised-bed boiler, taken from a real set of 18 written failure reports (the RCA case study). 13 reports are in the case library the agent can look up; 5 are held out for testing.
- **The six sensors:** `drum_level`, `feed_water_flow`, `steam_flow`, `drum_pressure`, `bed_temp_avg`, `ms_temperature`.
- **The data is simulated.** A physics simulator generates the episodes. There are 30 reporting episodes (used once per phase, to report) and 36 dev episodes (used for every decision).
- **The board:** Qualcomm QIDK HDK8650, Snapdragon 8 Gen 3. It has a CPU, a GPU and an NPU (Hexagon HTP v75), with 16 GB of memory shared by all three.
- **The research question:** how should an agent's model calls be scheduled across the NPU and CPU (and later GPU) when there is a deadline, no internet and limited power? Headline metric: energy per correct diagnosis. This part has not started yet. Everything so far builds what it needs.

Today the agent itself runs on a laptop. Only the language-model calls go to the board, over USB. Moving the whole agent onto the board is planned for Phase 4.

The single agent (the baseline)

This is the working system in `fieldmind/agent/`. Each tick runs a fixed pipeline. Two rules hold everywhere: the model never sees raw numbers, and the model never has the last word.

Step	What it does
Checks	Plain code turns sensor readings into **facts** with IDs (F1, F2, ...).
Triage	QUIET / WATCH / INVESTIGATE / URGENT. Decides whether the model is called at all.
Retrieval	Finds the library cases whose pattern matches what the sensors are doing.
Belief	A running score per cause, updated every tick by plain code.
Diagnostician	Model call 1: rank the causes and cite fact IDs.
Verifier	Model call 2: check the diagnosis against the facts.
Gate	Plain code checks the citations, picks actions from a fixed list of 16, decides escalation.
Memory	Carries the picture of the plant to the next tick.

The trouble with it: one big prompt (about 1,900 to 2,150 tokens), two calls one after the other on one processor, while the other processor sits idle. That is what the multi-agent design fixes.

2. Words used in this document

Word	Meaning
Tick	One 30-second step of the monitor.
Mock backend	A fake model that answers instantly from fixed rules. It lets us test all the code without a board. Mock numbers say nothing about how good a real model is.
Dev set / reporting set	36 episodes for making decisions / 30 episodes that are run once per phase only to report. We never tune on the reporting set.
GGUF	The file format llama.cpp uses for a model.
Quantization (Q4_0, Q8_0)	Storing each weight in 4 or 8 bits instead of 16, so the model is smaller and faster. Q4_0 and Q8_0 are two storage layouts.
"Pure" file	A GGUF where every weight matrix is Q4_0 or Q8_0 and nothing else.
llama-server	A llama.cpp program that loads a model once and then answers requests over HTTP until it is stopped. No reload between calls.
Lane	One running llama-server. NPU lane on port 8080, CPU lane on port 8081. Each lane runs one call at a time.
HTP0	llama.cpp's name for the NPU. "All layers on HTP0" means the model really runs on the NPU.
Prefill / decode	Prefill = the model reading the prompt (fast on the NPU). Decode = the model writing the answer, one token at a time (slow everywhere).
Context window	The most tokens (prompt plus answer) a server will hold. We launch with 4096.
Blackboard	The shared state the agents read and write. Each section has exactly one writer.

Word	Meaning
Gate (of a phase)	The list of checks a phase must pass, written down before building.
Group top-1	The model's first choice is in the right group of look-alike cases. This is the main accuracy number, because 9 of 13 cases look alike on 6 sensors.
Faithfulness	The share of facts the model cites that really exist.
Campaign	The long unattended board run: screening, dev rounds, model choice, reporting run.

3. How the work is organised

	Mac	Fedora laptop
Repo path	`~/projects/fieldmind`	`~/fieldmind`
Branch	`multi-agent`	`main`
Board attached	No	Yes (QIDK over USB, adb id `ab8f6592`)
Work	Building the multi-agent system against the mock	Real models on the board

GitHub: `git@github.com:Gagan-453/field-mind.git`

How we build

The code is written by Claude Code inside VS Code. A separate Claude chat is used to check every plan and every report before Claude Code continues: recompute the numbers, check the plan against the rules below, and write the next message to send. The session prompts are in `PROMPTS.md`, and the project rules are in `CLAUDE.md`. Claude Code is not allowed to `git push`; a person pushes.

The rules (please keep to these)

1. One change per commit, measured after each, so every number change has one cause.
2. Write the keep/revert rule before seeing results.
3. No tuning on the 30 reporting episodes. Decide on dev; run the reporting set once per phase.
4. A human overruling a rule is recorded as a HUMAN DECISION, with the reason.
5. Every number that moves needs a stated cause, or is marked "unexplained".
6. Do not change case data or thresholds to make numbers look better.
7. Every new test gets a mutation check (break the code on purpose; the test must fail), run with `PYTHONDONTWRITEBYTECODE=1`.
8. `fieldmind/agent/` is the baseline. The multi-agent branch must not change its behaviour.

4. The course of the project so far

Step	What happened
Session 0: baseline	Single agent measured on all 30 episodes with the mock. Tag `single-agent-v1`.
Session 1: fixes	Fixed the belief calculation. Library belief group score went 0.028 to 0.574. Tag `phase0a`.
Session 1b: measurement	Made the dev set (36 episodes). Fixed how confidence is shown (display only, no decision changes). Baseline saved as `results/baselines/single_v3_summary.json`. Tag `single-agent-v3`.
Session 2 prep	Code to talk to llama-server (`LlamaServerBackend`), the model-choice rule (`bench/model_choice.py`) and the GGUF checker (`bench/gguf_types.py`), committed before any real model ran (`ca93ed5`).
3 Oct, Fedora	Built llama-server for the board, built the 4 model files, smoke test passed, built the campaign runner, launched the campaign. Section 5.
3 Oct, Mac	Multi-agent Phase 1 finished and tagged. Phase 2 started. Section 6.

Earlier attempts to run a model on the board, and why we moved on:

Way of running	Model	Result
LiteRT-LM on CPU	Gemma 3 1B	Works, but 45 to 52 s per call.
LiteRT-LM on NPU	Gemma 3 1B	Context fixed at 1280 tokens. Our prompt did not fit.
QAIRT compiled graph, NPU	(context 2048)	Fastest, but the prompt had to be cut.
llama.cpp + Hexagon, NPU, old CLI	Llama 3.2 3B	7.1 s per call, but it reloaded the 1.79 GB model on every call.
llama.cpp + Hexagon, NPU, llama-server	the 4 candidates	What we use now. Model stays loaded; context set at launch.

5. Linux and the board: what is going on

5.1 What was done today

1. **Built llama.cpp for the board**, pinned at tag `b11371` (commit `99b9548`), with `llama-server` included. The package is on the board at `/data/local/tmp/llm/llama.cpp` (134 files, each checked by sha256).
2. **Built the 4 candidate model files with one recipe** (next section), checked each one, and pushed them to `/data/local/tmp/llm/` with sha256 checks.
3. **Smoke test on Llama 3.2 3B: passed.** 29 of 29 layers on the NPU, including the final output step. About 899 tokens/s reading the prompt and about 16.2 tokens/s writing the answer.
4. **Built the campaign runner** (`scripts/board_campaign.sh`), with 135 tests passing and 23 mutation checks caught.
5. **Launched the campaign** at 18:37. It failed, was diagnosed and fixed, and is being restarted clean. Section 5.5.

5.2 The four models and how the files were made

We do not download ready-made "Q4_0" files. A file with that name only means most of its weights are Q4_0; the Llama file we used earlier also held other types. So that every model is treated the same, all four were built from the original 16-bit weights with one recipe: every weight matrix Q4_0, token embedding and output layer Q8_0.

```
llama-quantize --pure --token-embedding-type q8_0 --output-tensor-type q8_0
```

Model	Source of the weights	File size	Check
Llama 3.2 3B Instruct	`unsloth/Llama-3.2-3B-Instruct` (mirror; Meta's repo is gated)	2.01 GB	196 matrices Q4_0, embedding Q8_0
Qwen3 1.7B	`Qwen/Qwen3-1.7B` (official)	1.46 GB	196 Q4_0, embedding and output Q8_0
Gemma 3 1B QAT	`unsloth/gemma-3-1b-it-qat` (mirror; Google's repo is gated)	0.72 GB	182 Q4_0, embedding Q8_0
Qwen2.5 0.5B Instruct	`Qwen/Qwen2.5-0.5B-Instruct` (official)	0.35 GB	168 Q4_0, embedding Q8_0

- The files are on the Fedora laptop in `~/fieldmind-build/gguf/` and on the board. **They are not in git** (too big). The build script is: `bench/build_candidate_gguf.sh`. The exact sizes and sha256 values are in `reports/phase0b_board_setup.md`.
- The mirrors were reported to hold weights byte-identical to the official files. We should still request official access to the Meta and Google repos so the licences are accepted in our name.
- The server refuses to load any model file that is not on the committed list with the right sha256, so the old impure file cannot be used by mistake.

Every lane is launched with the same flags: `-c 4096 -np 1 -fit off -lv 4 --cache-ram 0`. That is a 4096-token context, one request at a time, no automatic changes by the server, detailed logging (needed to see the NPU offload lines; measured cost was under 2%), and the server's prompt cache switched off (added tonight; section 5.5 explains why).

5.3 How the model is chosen

The rule was committed in `ca93ed5` before any real model ran, and the code applies it without a person choosing. It is in `bench/model_choice.py`.

Step 1. A model is out if it fails any of these:

- Every weight matrix is Q4_0 or Q8_0.
- All layers run on the NPU (`HTP0`).
- Broken JSON answers: at most 0.10 of first replies, and at most 0.02 after one repair try.
- Projected time for a verified diagnosis is at most 10 s, where the projection is `(700/prefill + 60/decode) + (350/prefill + 30/decode)`. That is one diagnosis call (700 tokens in, 60 out) plus one verifier call (350 in, 30 out), which are the call sizes of the multi-agent design. With the smoke-test rates, Llama 3B comes to about 6.8 s.

Step 2. Among the models that pass: the best library group top-1 on dev wins. If two are within 0.05, faithfulness decides. If those are within 0.02, the mean call time decides.

Step 3. Warnings that are recorded but do not stop the run: a lead that comes from a single episode, and a model scoring below what plain code achieves without any model (the "floor"; 0.501 group top-1 on the whole dev set).

Only two models get the full test ("option C")

Board time is limited. All 4 models are screened briefly. Full dev runs are done only for **Llama 3.2 3B** and **one other model**: the one that passes screening with the lowest measured time.

In the first (aborted) attempt that other model was **Qwen2.5 0.5B**, the smallest one, because the pick is by speed alone. The fresh campaign screens again, and the same result is expected. The pick is written to `results/board/A/screening.json`.

This leaves Gemma 3 1B and Qwen3 1.7B untested on real episodes. A follow-up dev run for them can be planned later, with its rule written first.

5.4 What exactly the campaign runs

Stage	What runs	Time (estimate)
A. Screening	All 4 models, one at a time: file check, NPU offload check, then 5 calls at 700 tokens in / 60 out and 5 calls at 350 / 30. Picks the second model.	about 10 minutes (it took 8 in the first attempt)
B. Dev rounds	Llama 3B and the second model, 3 dev episodes each, taking turns episode by episode. After round 1 a model is dropped if more than 0.30 of its first replies are broken JSON, or more than 0.10 stay broken after repair.	about 2 h
C. Model choice	The rule in 5.3, applied by code.	seconds
D. Reporting run	The chosen model on all 30 reporting episodes. This gives the **real-model baseline** that all multi-agent results will be compared with.	about 9 h
E. Ranking follow-up	13 dev episodes, only if round 3 leaves one question open (should belief or the model decide the order of causes).	about 4.5 h

- Settings: temperature 0, fixed seed, no JSON grammar, prompt caching off, one episode at a time, nothing else on the board. Wi-Fi off on the laptop when the run is meant to prove offline operation; the network state is logged for every episode.
- Before each episode the runner also logs the server's memory and the board's free memory. If free memory is under 2 GiB it restarts the server before that episode (never inside one) and logs it. Peak chip temperature is recorded per episode.
- Before each episode the runner waits until the chip is within 5 °C of its idle temperature (at most 15 minutes).
- A lost connection, a server crash or a call over 120 s is an **infrastructure failure**. It is never scored against the model. The runner retries 3 times, then stops cleanly.
- Results are written to `results/board/` and committed locally after each stage. Progress is in `results/board/STATUS.md`.
- The whole campaign was estimated at about 14 to 15 hours of running time. This estimate comes from call counts with the mock at 13.6 s per call, not from a measurement.

5.5 What happened tonight, and where it stands

Time	Event
18:37	First campaign launched. Screening finished in about 8 minutes.

Time	Event
19:11	Launch 1 died in the first dev episode. A power cut to the board was seen at about this time; the server was also already slowing down, so we cannot say which of the two stopped it.
19:14	Launch 2. It finished round 1 for both models, then died in `dev_B01_tube_leak`: a call got no answer in 120 s.
about 20:08	The board rebooted by itself, with no campaign running. Unexplained.
21:21	Launch 3 (through the restart wrapper). Died in the same episode, the same way, ten minutes in.
22:10	Diagnosis by Claude Code from the logs: two separate problems (below).
22:50	Soak test on the board confirmed the cause. Fixes committed.
23:10	Fresh campaign being launched, from screening, to run overnight.

Problem 1: the model server filled the board's memory

After every call, llama-server kept a copy of that prompt's working state in RAM (its "prompt cache"), even though we never reuse a prompt and ask it not to. For Llama 3B that is about 190 MB per call. After 31 to 37 calls the board's free memory reached zero, calls slowed down, and one never came back. The small Qwen2.5 0.5B needs far less per call and was not affected.

The fix: every server is now started with one more flag, `--cache-ram 0`, which switches that cache off.

The soak test sent the same 50 real prompts twice, changing only that flag:

	Run A: as before	Run B: with `--cache-ram 0`
Calls completed	36 of 50, then a stall	50 of 50
Longest call	over 300 s	18.5 s
Server memory	512 MB up to 6,598 MB	512 MB up to 535 MB
Board free memory	3.5 GB down to 0	6.29 GB down to 6.24 GB
Writing speed	16 to 17 tokens/s, falling to 11	15.8 to 16.3 tokens/s throughout
Peak CPU temperature	72 °C	67 °C

Heat alone did not cause the stall: both runs were at similar temperatures. The pre-set pass rule for run B said its memory must be "not growing"; it grew by 23 MB (0.5 MB per call), so by the letter the test did not pass. Accepting the result anyway was a HUMAN DECISION, recorded as made after seeing the result.

Problem 2: a bug in our runner

A timeout is meant to be retried and then end the run cleanly with exit code 2 and a stop reason in `STATUS.md`. Instead the program crashed with exit code 1, with no retries, and the status file went on saying "running". The cause: the runner file was loaded twice under two names, so the retry code was listening for a different copy of the error.

Fixed, with tests that run the real command line with an injected fault. Two smaller defects were fixed too: each attempt's call log is now kept (a relaunch used to erase the previous one), and the attempt number in the status file is now right. 144 tests pass.

- **The first attempt's results are archived** in `results/board_aborted_20261003`, with a note that they must never be used. The Llama 3B calls in it were timed on a server that was slowing down, and the server flags have changed since, so every number has to come from one configuration.
- **The fresh campaign starts from stage A.** Whether it was started at 23:10, and how far it got, is not known to this document.
- If overnight monitoring by Claude Code was set up, its notes are in `reports/campaign_watch.md`. In that case Wi-Fi stayed on, and the run does not count as the offline proof.

To see whether a campaign is really alive, use all three: the tmux session exists, the last lines of the log show no error, and a file changed in the last few minutes.

```
tmux ls
scripts/board_campaign.sh status
tail -5 logs/campaign_loop.log
find results/board logs -type f -mmin -5
```

5.6 How to launch or restart it

Before launching: board and laptop on mains power, lid open, Wi-Fi off, `adb devices` shows `ab8f6592`, the chip near its idle temperature (about 33 °C), and about 10 minutes since the board booted. First make sure no campaign is already running. Plain launch:

```
tmux new -s campaign 'systemd-inhibit --what=sleep:idle scripts/board_campaign.sh all'
```

Because the power is unreliable, this wrapper is better. It waits for the board, lets it settle 10 minutes, launches, and relaunches by itself after a power cut (exit code 2 only), up to 12 times. Save it as `~/campaign_loop.sh`:

```
#!/usr/bin/env bash
cd ~/fieldmind || exit 1
for i in $(seq 1 12); do
  echo "$(date -Is) launch $i: waiting for board" | tee -a logs/campaign_loop.log
  adb wait-for-device
  sleep 600
  adb shell svc power stayon usb
  scripts/board_campaign.sh all 2>&1 | tee -a logs/campaign_loop.log
  code=${PIPESTATUS[0]}
  echo "$(date -Is) exit code: $code" | tee -a logs/campaign_loop.log
  [ "$code" -eq 2 ] || break
done
```

```
chmod +x ~/campaign_loop.sh
tmux new -s campaign 'systemd-inhibit --what=sleep:idle ~/campaign_loop.sh; exec bash'
```

- Leave the tmux window with `Ctrl-b` then `d`. Come back with `tmux attach -t campaign`. Do not press `Ctrl-c` inside it.
- Exit codes: **0** finished, **2** infrastructure failure (safe to relaunch), **3** stopped by one of our rules (do **not** relaunch; read the status and the tables first).
- The wrapper starts the campaign correctly. Its automatic relaunch has never happened for real, so that part is untested. It relies on the board switching itself on when power returns.
- Exit code 1 means a crash in our own code. Do not relaunch; read the end of `logs/campaign_loop.log`.

6. The Mac: the multi-agent system

6.1 The idea

Instead of one big prompt answered by one model call at a time, the work is split into small agents. Some are plain code and run every tick. Some call the model, with short prompts, and can run on the NPU and the CPU at the same time. The tick never waits for a model: plain code publishes its own assessment every tick, and model answers are added when they arrive. The full design is in `docs/multi_agent_plan.pdf`.

Phase	What it is
0	Fix the single agent. **Done** (on `main`).
1. Plumbing	Build the multi-agent machinery, but make it give exactly the same decisions as the single agent. **Done, tagged `multi-phase1`.**
2. Prompt shrink	Make every prompt fit in 1280 tokens and every answer in 60. **In progress: 4 of 8 commits.**
3. Split by plant side	Two diagnosticians: water side on the NPU, heat side on the CPU. Test the verifier with a known-wrong answer. Not started.
4. Real time	Run in real time on the board; add the drift watcher; move the agent onto the board. Not started.
5. Scheduling study	Compare scheduling policies; measure energy; GPU numbers. This is the research contribution. Not started.

6.2 Phase 1: what was built

All new code is in `fieldmind/multi/`. It reuses the single agent's code by importing it, not by copying it.

- **Blackboard:** the shared state. Every section has exactly one writer; a write from the wrong agent raises an error.
- **Agents:** sensor, triage, retriever + belief, diagnostician, verifier, gate + memory. Same order as the single agent.
- **Scheduler:** priorities P1 to P4, at most one waiting job per agent.
- **Lanes:** NPU and CPU, each running one job at a time on a simulated clock.
- **Fact IDs carry their tick** (`t84.F1`), because an answer can arrive a tick late and must be checked against the facts of the tick it was asked about.
- **How to run:** `run_demo.py --arch single|multi --mode lockstep`, with `--record-prompts` to save every prompt. Real-time mode is not built (Phase 4).

Phase 1 gate results (mock backend, laptop timings):

Check	Dev (36 episodes)	Reporting (30 episodes)
Decisions, multi vs single	0 differences in 5,850 ticks	0 differences in 5,160 ticks
Prompts sent to the model	byte-identical	byte-identical
Single agent on this branch vs `main`	0 differences	matches the v3 baseline
Deadline misses	0	0
Ticks where the code path took over 200 ms	0	0

Tests: 154 passed at the end of Phase 1, with 15 deliberate code corruptions all caught. The report is `reports/multi_phase1_plumbing.md` and the saved summary is `results/baselines/multi_p1_summary.json`.

6.3 Phase 2: what is done

Phase 2 is mock-only for now. It can prove prompt sizes, parsing and unchanged decisions. It **cannot** say anything about accuracy until the real-model baseline from the campaign exists.

Commit	What it holds
0 (`595180b`)	The decisions and the gate (G1 to G6), written before any measurement.
1 (`c7ddcc0`)	`bench/time_single_hard.py`: the timing script for the single agent's code path.
2 (`a36e4a8`)	**Fixed placement** in the scheduler: each agent always uses its assigned lane. It is now the default. Diagnostician on NPU, verifier and text reader on CPU. 0 decision differences against the old mode.
3 (`0d3254f`)	`bench/token_count.py`: counts tokens with the real tokenizers of all 4 models. Result: Phase 1 prompts are about 1,870 to 1,890 tokens on average and up to 2,150; 1,980 of 1,982 are over the 1,280 limit.

Full test suite after commit 3: 197 passing. The report is `reports/multi_phase2_prompt_shrink.md` and the step-by-step state is in `reports/multi_phase2_progress.md`.

6.4 Phase 2: decisions already taken

These were sent to Claude Code as HUMAN DECISIONS. Check the report to confirm each one was written down.

- **Allowed accuracy drop** (to be measured later with the real model, on dev): group top-1 may drop by at most 0.05, faithfulness by at most 0.02, and group top-1 must stay above the floor (0.501).
- **Answer format:** the planned ID-only answer did not fit in 60 tokens on any tokenizer (it measured 89 to 135). So the model now answers with **line numbers** of the cases, facts and notes shown in the prompt, with no confidence numbers. Worst case 49 tokens. Code maps the line numbers back to real IDs, using a map saved when the prompt was built.
- **At most 9 fact lines** per diagnosis prompt, most severe first. Dropped facts are logged.
- **Verifier answers pass or fail for every case shown.** A case with no verdict counts as "not judged", never as a pass.
- **New faithfulness measure, Q3_rel.** With line numbers, "the cited ID exists" is nearly always true and stops measuring anything. Q3_rel counts a citation only if that fact really supports that case according to the plain-code layer. The Phase 2 gate uses Q3_rel.

- **Confidence** for a case outside belief's top 3 comes from belief (capped at 0.5; 0.3 if belief has no entry). Predicted effect on dev: verifier calls go from 192 to 327.
- **Plant records are turned into facts by code**, with no model call. The text reader (a new small model agent) handles engineer notes only.
- **The verifier loses three things** (a design change, not just a shrink): the check that a distinguishing sign is observable, the rewrite of the top confidence, and claims beyond rank 3.
- **Token limit:** prompt tokens plus that agent's answer limit must be under 1,280 on all four tokenizers. Answer limits: diagnostician 60, verifier 30, text reader 50.
- **Gate G1 is split in two:** G1a changes only the answer format and allows differences only in confidence-related fields, with the counts predicted in advance. G1b switches on all the prompt shrinking and must show 0 differences against G1a.

6.5 Phase 2: what is left

Commit	What it should hold
4	The text reader agent: one engineer note in, one checked note-fact out.
5	The shrunk diagnostician prompt, with a separate on/off switch for each shrunk section so a later failure can be traced to one section.
6	The shrunk verifier prompt.
7	The dev gate G1 to G6, then one reporting run, then the report.

We do not know how far Claude Code got after the last message. Read `reports/multi_phase2_progress.md` and `git log --oneline -12` on the Mac.

7. Pushing everything to GitHub

Claude Code cannot push, so these are typed by hand. Wait until Claude Code has stopped and is not in the middle of a commit.

7.1 On the Mac (branch multi-agent)

```
cd ~/projects/fieldmind
git status
git log --oneline -12
```

`git status` should be clean. If `episodes.tgz` or `episodes.sha256` are listed, move them out of the repo (`mv episodes.tgz episodes.sha256 ~/`). If there are real uncommitted changes, ask Claude Code to commit them with a message saying they are unfinished; do not use `git add -A` blindly. Then:

```
git push -u origin multi-agent
git push origin multi-phase1
```

The first line pushes the branch. The second pushes the Phase 1 tag. Check on GitHub that the branch `multi-agent` and the tag `multi-phase1` are there.

7.2 On the Fedora laptop (branch main)

Everything up to commit `586338f` (all the setup work and the campaign runner) was already pushed before the launch. What is only on the laptop is the campaign's result commits, starting with `afde367`.

```
cd ~/fieldmind
git status
git log --oneline origin/main..main
git push
```

- This includes tonight's fixes (the runner bug, `--cache-ram 0`, the memory guard) and the archived first attempt. Push them soon, so they are not only on one laptop.
- If the campaign is running with Wi-Fi off as an offline proof, wait until it finishes. Pushing itself does not disturb the run.
- The campaign makes its own commits after each stage. Push again when it is done.

7.3 Add this document to the repo

So that it arrives with the clone. Run this in the repo, on either machine, and change the first path to wherever you saved the PDF:

```
cp ~/Downloads/FieldMind_team_handoff_2026-10-03.pdf docs/
git add docs/FieldMind_team_handoff_2026-10-03.pdf
git commit -m "docs: team handoff, 3 Oct 2026"
git push
```

7.4 What is NOT on GitHub, on purpose

Thing	How a teammate gets it
Episodes (`data/episodes`, `data/episodes_dev`)	Regenerate them. The generator gives byte-identical files on every machine.
The 4 model files (4.55 GB)	Only on the Fedora laptop and the board. Not needed for mock work. Can be rebuilt with `bench/build_candidate_gguf.sh`.
Source weights (about 27 GB)	On the Fedora laptop in `~/fieldmind-build`. Keep them until the campaign is done.
Tokenizer files	`bench/token_count.py --fetch` downloads them; a committed manifest checks their sha256.
`npu/maven/` (Qualcomm libraries)	Must never be committed.
`data/experience/store.json`	Empty on both machines. Keep it empty, or prompts will differ between machines.

8. For teammates: getting started on your own laptop

First ask the repo owner to add your GitHub account as a collaborator. Then:

```
git clone git@github.com:Gagan-453/field-mind.git
cd field-mind
git switch multi-agent
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest
```

- If you have no SSH key on GitHub, clone with `https://github.com/Gagan-453/field-mind.git` instead.
- Always use `.venv/bin/python`, never the system Python.
- The tests should all pass before you change anything. If the count is lower than 197, check that you are on `multi-agent` and up to date.
- **Generate the episodes** by following `README_SETUP.md` in the repo. This document does not quote the command, because we have not checked it here.
- Then run the mock on both architectures and confirm you get 0 differences, as in section 6.2. The flags are `run_demo.py --all --backend mock --arch single` and `--arch multi --mode lockstep`; check `CLAUDE.md` for the exact form.

Read these, in this order: `CLAUDE.md` (rules and layout), `PROMPTS.md` (one prompt per session), `docs/multi_agent_plan.pdf` (the design), `reports/multi_phase1_plumbing.md`, `reports/multi_phase2_prompt_shrink.md` and `reports/multi_phase2_progress.md`.

> **Avoid stepping on each other**
>
> Only one person should have Claude Code working on the `multi-agent` branch at a time, or work on your own branch made from it (`git switch -c multi-agent-yourname`) and merge through a pull request.
>
> Run `git pull` before starting and push when you stop.
>
> Do not edit `fieldmind/agent/` or `bench/model_choice.py` on the multi-agent branch. The campaign on `main` depends on them.

9. What to do from here, in order

Step 1. Finish Phase 2 on the mock (any laptop)

- Read `reports/multi_phase2_progress.md` to see which of commits 4 to 7 exist.
- Continue with Claude Code using Session 4 in `PROMPTS.md`, plus the decisions in section 6.4. One commit per agent. Stop and review after each.
- Before the G1a run, the list of fields allowed to change and the predicted counts must already be committed. If a measured count differs from its prediction, stop; do not adjust the prediction.
- Pass the dev gate G1 to G6 first. Only then run the reporting set once.
- Still to answer: why 1,154 of 3,170 retrieved cases have no belief entry. Look, do not fix; if the cause is in `fieldmind/agent/`, record it for `main`.

Step 2. Get the campaign running again, then finish it (Fedora laptop)

- **Check on the fresh campaign** with the commands in section 5.5. The first thing to look for: Llama 3B finishing two dev episodes, since the first attempt always died before that.
- If it stopped: exit code 2 is safe to relaunch, exit code 3 is a rule stop (read the tables, do not relaunch), exit code 1 is a bug (read the log).

- Do not change the server flags, the 120 s timeout, the token limits or any committed rule while it runs. Any such change is a human decision, recorded with its reason, and means starting again so that all numbers come from one configuration.
- When it finishes:
- Read the model-choice table and the reporting summary. Check the chosen model against the committed rule by hand, and compare its group top-1 with the 0.501 floor (the runner prints only the floor it computed on the same 3 episodes).
- Look at the memory-guard and peak-temperature lines for each episode. They have not run in a real episode before. A CPU peak of 93.8 °C was seen in the first attempt and is unexplained.
- Then test whether replies are repeatable (section 10): about 25 minutes on the board.
- Push `main`. The real-model baseline is saved as `results/baselines/single_v3_real_<model>_npu_summary.json`.
- The source weights in `~/fieldmind-build` (27 GB) can be deleted once the model is settled.

Step 3. Join the two branches

```
git switch multi-agent
git pull
git merge origin/main
```

- Expect a conflict in `run_demo.py`; both branches changed it.
- After the merge, rerun the Phase 1 gate (0 differences between single and multi).
- Then: make `bench/model_choice.py` understand tick-stamped fact IDs; replace the placeholder lane speeds in the scheduler with the measured ones; correct the "about 2,484 tokens" reason text (see section 10).

Step 4. The real-model test of Phase 2 (needs the board)

Run the shrunk prompts with the chosen model on dev, with fixed placement, and compare with the real-model baseline using the limits in section 6.4. If it fails, switch the shrunk sections back on one at a time to find which one costs accuracy. Also measure, as separate numbers: how often the NPU and the CPU give different answers to the same prompt, and whether the model just answers "1, 2, 3" (use the case-order switch).

Step 5. Later phases

- **Phase 3:** split the diagnostician into water side (NPU) and heat side (CPU). Test the verifier with a deliberately wrong answer; so far it has never disagreed with anything.
- **Phase 4:** real time on the board, the drift watcher, and moving the agent onto the board. The write audit (a debug check that costs up to about 100 ms per tick on a laptop) must be taken out of the timed path first.
- **Phase 5:** the scheduling study, energy measurement and GPU numbers.

10. Things that are not verified, or still open

Item	State
Is the fresh campaign running?	It was being launched at about 23:10 on 3 Oct. Not confirmed here. Check with the commands in section 5.5.

Item	State
Are the model's replies repeatable?	**Open, and important.** At temperature 0 with a fixed seed, replies differed between the two soak runs on 10 of 37 calls. The server flags differed between those runs, so the cause is not known. Until it is tested with identical flags, we make no claim that results are reproducible across launches, and small differences between models should be read with that in mind.
Small memory growth with the cache off	0.5 MB per call over 50 calls. Whether it levels off is not known. The memory guard covers it.
93.8 °C CPU in the first attempt	Unexplained. The soak test peaked at 72 °C at the same stall.
Why the board rebooted at about 20:08	Unknown. The board recorded an ordinary reboot request; nothing on the laptop sent one.
What stopped launch 1 at 19:11	A power cut or the memory problem; cannot be told apart.
Which model is second in the dev rounds	Qwen2.5 0.5B in the first attempt. To be confirmed in the fresh run's screening file.
How far Phase 2 got on the Mac	Unknown after commit 3. Check the progress file.
Whether the Mac branch and tag are pushed	Not confirmed. Section 7.1.
"Other weight types fall back to the CPU"	Seen on the earlier build. Claude Code reported that the current llama.cpp source also accepts those types. Not tested on the board. The recipe stays the same for all 4 models, so the comparison is fair, but do not write this claim in the paper without a board test.
Old Llama 3B speeds (839 / 11.7 tokens/s)	Measured on the impure file, with a different program. The new 899 / 16.2 is not a controlled comparison with them.
"Single-agent prompt is about 2,484 tokens"	Not reproduced. The measured maximum is about 2,160. The 4096 context is still needed. The campaign logs the server's own token count per call, which will settle it.
Memory guard and peak temperature	New tonight. Tested against the fake board only; not yet seen in a real episode.
Runner's per-stage commit	Worked for stage A in the first attempt. Later stages not yet seen.
Gemma tokenizer	Our file gives the same token IDs as the Hugging Face tokenizer on 2,084 dev prompts. About 6,400 differing scores and types in the vocabulary are unexplained, with no effect on those prompts.
The restart wrapper's relaunch	Never exercised for real.
Timing outliers on the laptop	A 23.4 ms maximum in one run and a maximum that did not reproduce (0.78 vs 1.8 ms). Marked unexplained; they do not affect decisions.
Text reader quality	Unmeasured. The mock's note-facts come from code-built hints.
Energy and GPU	No numbers exist yet.
Official access to gated repos	Meta (Llama) and Google (Gemma) still to be requested.

Longer-standing open problems and the questions for Prof. Shukla are in the earlier handoff (`FieldMind_handoff_2026-10-03.pdf`, section 9) and are unchanged: the belief scale is over-confident, the verifier never disagrees, the hold-out covers only one case, and the drum level constant is unresolved.

11. Quick reference

Commands

What	Command
Is the campaign alive?	`tmux ls`
Campaign progress	`scripts/board_campaign.sh status`
Watch the campaign	`tmux attach -t campaign` (leave with `Ctrl-b` then `d`)
Is the board connected?	`adb devices`
Is a server running on the board?	`adb shell "ps -A | grep llama"`
Keep the board awake	`adb shell svc power stayon usb`
Run the tests	`.venv/bin/python -m pytest`

Commits and tags

Hash / tag	Branch	What
`single-agent-v3`	main	Single-agent baseline (mock).
`ca93ed5`	main	Model-choice rule, committed before any real-model run. The `multi-agent` branch starts here.
`a75bdc1`	main	The campaign runner.
`586338f`	main	Last commit pushed to GitHub before the launch.
`afde367`	main	Stage A results of the first, aborted attempt.
`b4f1936`	main	Soak test table and raw data, and the corrected report.
`10ce4df`	multi-agent	Single agent's tick code made importable (a human decision).
`0404b04`	multi-agent	Phase 1 gate, written before the code.
`multi-phase1`	multi-agent	Phase 1 finished.
`595180b` to `0d3254f`	multi-agent	Phase 2, commits 0 to 3.

Files worth knowing

File	What it is
`CLAUDE.md`, `PROMPTS.md`	Rules, layout, and the prompt for each session.
`docs/multi_agent_plan.pdf`	The multi-agent design.
`reports/phase0b_board_setup.md`	Everything about the board setup and the runner.
`reports/session2_setup_progress.md`	Step-by-step state of the Fedora work.
`reports/multi_phase1_plumbing.md`	Phase 1 report.
`reports/multi_phase2_prompt_shrink.md`	Phase 2 decisions, gate and results so far.
`reports/multi_phase2_progress.md`	Step-by-step state of Phase 2.

File	What it is
`results/board/STATUS.md`	Live campaign status.
`results/baselines/`	Saved baseline summaries.
`bench/model_choice.py`	The model-choice rule as code.
`bench/gguf_types.py`	Checks that a model file is pure.
`bench/token_count.py`	Counts tokens with the real tokenizers.
`scripts/board_campaign.sh`	The campaign runner.